

Q&A Preparation

Likely audience questions and short answers

Compensating Connectivity Restrictions in Quantum Annealers via Splitting and Linearisation Techniques

Seelbach Benkner, Löhner, Golyanik, Kliesch, Moeller arXiv:2507.12536v1 (2025)

CENG 591 Graduate Seminar
Denizcan Yilmaz 2444172

Thirty short questions a graduate audience might ask after the talk, with concise answers at the level of the speaker notes. The answers stay within what was actually said or shown; for anything beyond that, the safest reply is to acknowledge the limit of the talk and gesture at the paper. Read this once before the session to internalise the wording.

A. Background and motivation

Q1. What does QUBO stand for and why is it useful?

A. *Quadratic Unconstrained Binary Optimisation. It is a standard way to write many combinatorial problems as minimising $x^\top Qx$ over binary variables, and quantum annealers solve exactly this form natively.*

Q2. What is the difference between QUBO and the Ising model?

A. *They are mathematically equivalent. QUBO uses binary variables in $\{0, 1\}$ and Ising uses spins in $\{-1, +1\}$. The conversion is just $s = 2x - 1$. Quantum hardware works with spins, so the paper writes everything in the Ising form.*

Q3. What is the connectivity restriction in physical terms?

A. *Each qubit can only interact with a small fixed set of physical neighbours, set by the chip wiring. If the problem needs a coupling that the hardware does not wire, it cannot be encoded directly.*

Q4. Why does a 5,000-qubit chip only fit dense problems with around 150 variables?

A. *A dense problem needs an interaction between every pair of variables. To make those interactions, the standard fix glues many physical qubits into a chain per logical variable, and the chain overhead burns through the 5,000-qubit budget very quickly.*

Q5. What kinds of problems does this paper target?

A. *Combinatorial optimisation problems that can be written as a QUBO: scheduling, vehicle routing, circuit layout, graph cuts, and similar NP-hard tasks where classical solvers struggle at scale.*

B. Minor embedding

Q6. What is minor embedding in plain terms?

A. *It is the process of mapping a problem graph onto the hardware graph by representing each logical variable as a chain of several physical qubits, with a strong penalty term that forces the chain to agree.*

Q7. Why is finding a minor embedding hard?

A. *It is an NP-hard graph problem in general, with no efficient algorithm known. The standard heuristics still take minutes per instance, and they often fail entirely for large or dense problem graphs.*

Q8. When would you still prefer minor embedding over splitting?

A. *For small instances where the embedding fits and you can afford the preprocessing time, embedding tends to give a more accurate solution because it does not introduce any approximation.*

C. The splitting idea

Q9. Why split A based on the hardware graph specifically?

A. *Because some interactions in A already have a physical wire on the chip and can be handled exactly. The others need to be approximated. The split $A = A_Q + A_L$ isolates the exact part from the approximated part.*

Q10. What goes into A_Q and what goes into A_L ?

A. A_Q holds the edges already on the hardware, where the adjacency matrix M has a one. A_L holds the remaining edges, which the hardware cannot wire and therefore have to be approximated.

Q11. Why does the linearisation result in a per-qubit bias?

A. Because a first-order Taylor expansion turns the quadratic $s^\top A_L s$ into something linear in s , that is, a fixed bias per qubit. A linear term only acts on individual qubits, not pairs, so the hardware can apply it directly without any new coupling wires.

Q12. What does the damping term do?

A. It penalises jumps where the Taylor approximation would break down. On the discrete cube $\{-1, +1\}^n$, the squared-norm penalty itself reduces to a per-qubit bias, so it also fits the hardware directly.

Q13. Why apply a random permutation each iteration?

A. Without it, the same edges always get linearised, so the approximation error builds up systematically in the same places. Permuting reshuffles which edges land on hardware, so over many iterations every edge gets the exact treatment some of the time.

D. Algorithm and parameters

Q14. What controls the value of λ ?

A. Theoretically, weak monotonicity is guaranteed whenever $\lambda \geq \|2A_L\|$, the spectral norm. In practice the paper also tries smaller values to allow occasional uphill moves and escape local minima.

Q15. What is the “force vector” you mentioned?

A. It is a vector whose sign at each entry tells you whether the corresponding variable would flip at the next iteration. Only sign changes matter, which is why only certain λ values produce different outcomes.

Q16. How many λ values does the algorithm actually test?

A. The default is fifteen candidates per iteration, taken from the midpoints between sorted entries of the force vector. The lowest-energy iterate among them is kept.

Q17. How is the algorithm initialised?

A. With a random assignment of ± 1 to all variables. There is no warm start from another method.

E. Theory and proof

Q18. What does weak monotonicity actually guarantee?

A. Only that the true energy never increases from one iteration to the next. It does not guarantee reaching the global minimum.

Q19. What is the surrogate function in the proof?

A. It is $E^{(k)}(x)$: the energy with A_L linearised at $s^{(k)}$ plus the damping term. By the Taylor remainder theorem, for λ large enough it is a safe upper bound on the true energy at every point.

Q20. Why does convergence in finitely many steps follow from monotonicity?

A. *Because there are only 2^n possible binary vectors, and the energy is non-increasing. We can only descend so many times before we run out of distinct states.*

Q21. What is the relationship to proximal gradient descent?

A. *The iteration is mathematically equivalent to the proximal gradient descent algorithm applied to a composite problem $\min f(s) + g(s)$, where f is the linearised part and g is the hardware-friendly part plus the discrete ± 1 constraint.*

F. Comparison to LNLS and other methods

Q22. How does splitting compare to LNLS?

A. *LNLS, the Large Neighbourhood Local Search competitor, fixes most variables and optimises the rest exactly. Splitting linearises the rest instead. Empirically they are comparable, with splitting matching LNLS at sub-problem size around thirty.*

Q23. When does splitting beat LNLS?

A. *Mostly on dense or fully-connected graphs, where the global linearisation captures structure that small fixed neighbourhoods cannot. On very sparse graphs, specialised heuristics that exploit the sparsity often outperform splitting.*

Q24. Is there a clear rule for when splitting is the right choice?

A. *The paper is honest that no reliable predictor was found. The recommendation is essentially to try it, especially on dense problems too large for minor embedding.*

G. Experiments and figures

Q25. What datasets were used in the experiments?

A. *Two: a regular spin-glass dataset of 270 instances with sizes from ten upward, and 909 MQLib Max-Cut instances with 500 to 1,000 variables.*

Q26. What does “energy divided by best heuristic” mean on the y-axis?

A. *It is a normalised approximation ratio: the obtained energy divided by the best energy a reference heuristic could find. Higher means better, with 1.0 corresponding to matching the best known solution.*

Q27. What was the wall-clock advantage over minor embedding?

A. *Roughly two orders of magnitude on the spin-glass instances. Minor embedding alone takes up to ten minutes of preprocessing, while splitting and LNLS finish in well under a second.*

H. Limitations and extensions

Q28. Why does the method fail on float-valued or wide-range integer weights?

A. *D-Wave’s coupling strengths have limited numerical precision. Very large or very fine-grained weights cannot be represented faithfully, so the energy landscape the chip sees is distorted.*

Q29. Could this idea generalise to gate-based quantum computers?

A. *The splitting framework itself is generic, but the implementation here relies on a hardware-restricted QUBO solver. Adapting it to a gate-based device would require a different sub-solver, and the paper does not test this.*

Q30. What is the simulated-annealing extension that is mentioned?

A. A sketch in Section 3.8 adds a temperature-dependent stochastic term so the algorithm can take occasional uphill moves and escape local minima. It is theoretically motivated but the paper leaves a full numerical evaluation to future work.

I. Symbol & variable glossary

Quick reference for the symbols that appear in the slides and equations. Use this if asked “what does X stand for?” during questions.

Problem variables

n	Number of variables in the optimisation problem.
x	Binary variable vector in $\{0, 1\}^n$ (QUBO form).
s	Spin variable vector in $\{-1, +1\}^n$ (Ising form). Related to x by $s = 2x - 1$.
$\hat{x}, \hat{s}$	The optimal (cost-minimising) assignment we are trying to find.

Coupling matrices and biases

Q	QUBO coupling matrix, $n \times n$. The entry Q_{ij} encodes how variables i and j interact in the cost.
A	Ising pairwise coupling matrix, $n \times n$. Related to Q by $A = Q/4$.
b	Ising external bias (field) vector, $b = Q\mathbf{1}/2$, where $\mathbf{1}$ is the all-ones vector.
A_Q	The “quadratic” part of A : only entries where the hardware has a physical wire. Handled exactly by the annealer.
A_L	The “linearised” part of A : the remaining entries that the hardware cannot wire. Approximated by linearisation.

Hardware and operations

M	Hardware adjacency matrix. $M_{ij} = 1$ if there is a physical wire between qubits i and j , otherwise 0.
P_k	Random permutation matrix drawn fresh at iteration k . Reshuffles the mapping of problem variables to physical qubits.
$P_k^\top$	The transpose (= inverse, since P_k is a permutation) of the permutation matrix.
$\odot$	The Hadamard product, that is, element-wise multiplication of two matrices of the same shape.

Iteration and algorithm

$s^{(k)}$	The current iterate (the spin vector after iteration k).
$s^{(k+1)}$	The next iterate, produced by solving the hardware-friendly subproblem.
λ	Damping (regularisation) strength. Larger λ = smaller, safer steps. Monotonicity is guaranteed when $\lambda \geq \ 2A_L\ $.
$\ \cdot\ $	Spectral norm of a matrix (the largest singular value, equivalent to the largest absolute eigenvalue for symmetric matrices).
L	Linear force vector used to select λ candidates. Each entry’s sign tells whether the corresponding variable would flip next step.

Energy and proof

$E(s)$	The true energy at spin assignment s , equal to $s^\top A s + s^\top b$. We want to minimise this.
$E^{(k)}(x)$	The surrogate energy at iteration k : the true energy with A_L replaced by its linearisation, plus the damping term.
$f(s)$	In the proximal gradient view: the smooth part, $f(s) = s^\top A_L s$. Linearised in the iteration.
$g(s)$	In the proximal gradient view: the non-smooth part, $g(s) = s^\top A_Q s + \mathbb{I}_{\{-1,+1\}^n}(s)$. Solved exactly by the hardware.
$\mathbb{I}_{\{-1,+1\}^n}$	The indicator function of the discrete cube: 0 if $s \in \{-1,+1\}^n$, $+\infty$ otherwise. Encodes the binary constraint.
■	Standard end-of-proof marker.

End of Q&A preparation. If a question goes deeper than the talk, it is fine to say “the paper does not go into that in detail” rather than guess.